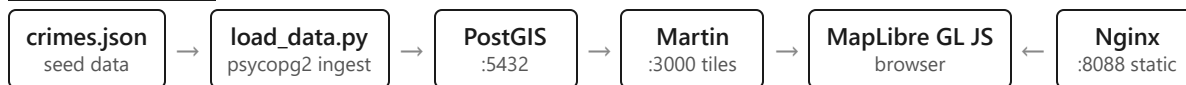


Open-Source Mapping Stack

PostGIS · Martin · MapLibre GL JS · Nginx · Podman

ARCHITECTURE



QUICK START

```
cp .env.sample .env
python3 -m pip install -r requirements.txt
./start.sh
```

`start.sh` builds and starts all three containers, then runs `load_data.py` to seed PostGIS.

ENVIRONMENT (.ENV)

```
POSTGRES_DB=kenda_cases
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
DATABASE_URL=postgres://postgres:postgres\
@localhost:5432/kenda_cases
```

LIVE ENDPOINTS

Service	URL
Web map	http://localhost:8088
Martin TileJSON	http://localhost:3000/crimes
Vector tile	http://localhost:3000/crimes/{z}/{x}/{y}
Martin catalog	http://localhost:3000/catalog
PostGIS	localhost:5432 (psql / QGIS)

USEFUL COMMANDS

```
# Inspect running containers
podman ps

# Open a psql shell
psql $DATABASE_URL

# Count loaded cases
psql $DATABASE_URL -c "SELECT count(*) FROM crimes;"

# Check spatial index
psql $DATABASE_URL -c "\d crimes"

# Re-run loader after editing crimes.json
python3 load_data.py
```

POSTGIS SCHEMA (KEY COLUMNS)

```
id          SERIAL PRIMARY KEY
title       TEXT NOT NULL
year        INTEGER
crime_type  TEXT
solved      BOOLEAN
lat / lon   DOUBLE PRECISION
geom        GEOMETRY(Point, 4326) — spatial index via GIST
```

```
-- Geometry is computed on insert:
ST_SetSRID(ST_MakePoint(lon, lat), 4326)
```

MARTIN CONFIG (MARTIN.YAML)

```
postgres:
  connection_string: ${DATABASE_URL}
  auto_publish:
    tables: true
    from_schemas: public
  listen_addresses: "0.0.0.0:3000"
```

Martin auto-discovers and publishes every table in the `public` schema that has a geometry column — no additional config needed when you add a new table.

MAPLIBRE GL JS (APP.JS)

Basemap OpenFreeMap dark style (no API key)
Source `type: "vector"` → Martin tile URL
Layers circle halo + filled dot, color-coded by `solved`
Sidebar Loaded from local `crimes.json` (filter / stats)
Popups On click: title, victim, year, weapon, description

CONTAINER IMAGES

PostGIS [docker.io/postgis/postgis:18-3.6](https://hub.docker.com/r/postgis/postgis)
Martin ghcr.io/maplibre/martin:latest
Nginx standard nginx, serves `/usr/share/nginx/html`

KEY DEPENDENCIES (REQUIREMENTS.TXT)

```
psycopg2-binary # PostgreSQL adapter for Python
```

Tip: Connect QGIS directly to PostGIS on `localhost:5432` to inspect geometries, run spatial queries, or style layers independently of the web map.

Open-Source Mapping Stack — PostGIS Queries

queries.sql · connect: psql \$DATABASE_URL

1 · ROW COUNTS

```
SELECT
  count(*)                               AS total,
  count(*) FILTER (WHERE solved)         AS solved,
  count(*) FILTER (WHERE NOT solved)     AS unsolved
FROM crimes;
```

2 · ST_ASTEXT — SEE THE GEOMETRY

```
SELECT title, ST_AsText(geom) AS wkt
FROM crimes LIMIT 5;
```

Converts the binary geom column to human-readable WKT coordinates.

3 · ST_ASJSON — EXPORT A ROW

```
SELECT title, ST_AsGeoJSON(geom) AS geojson
FROM crimes WHERE id = 1;
```

Same format MapLibre and most web APIs consume directly.

4 · ST_DWITHIN — RADIUS SEARCH

```
-- Crimes within 1 mile of downtown Colorado Springs
SELECT title, year, solved
FROM crimes
WHERE ST_DWithin(
  geom::geography,
  ST_MakePoint(-104.8214, 38.8339)::geography,
  1609 -- metres (1 mile)
)
ORDER BY year;
```

::geography cast → PostGIS works in metres; no projection math needed.

5 · KNN — NEAREST NEIGHBOURS

```
SELECT title,
  round(ST_Distance(
    geom::geography,
    ST_MakePoint(-104.8214, 38.8339)::geography
 )::numeric) AS dist_m
FROM crimes
ORDER BY geom::geography
<-> ST_MakePoint(-104.8214, 38.8339)::geography
LIMIT 5;
```

<-> is the KNN distance operator — index-accelerated via GiST.

6 · AGGREGATE BY DECADE

```
SELECT (year/10*10)::text||'s' AS decade,
  count(*) AS cases,
  round(100.0
    * count(*) FILTER (WHERE solved)
    / count(*)) AS pct_solved
FROM crimes
GROUP BY decade ORDER BY decade;
```

7 · ST_BUFFER — ZONE OF INTEREST

```
WITH zone AS (
  SELECT ST_Buffer(
    ST_MakePoint(-104.8214, 38.8339)::geography,
    2000 -- metres (2 km radius)
  )::geometry AS circle
)
SELECT count(*) AS crimes_within_2km
FROM crimes, zone
WHERE ST_Within(geom, zone.circle);
```

8 · ST_CENTROID — GEOGRAPHIC CENTER

```
SELECT ST_AsText(
  ST_Centroid(ST_Collect(geom))
) AS center_of_all_crimes
FROM crimes;
```

ST_Collect aggregates all points; ST_Centroid finds their geographic center.

GOING FURTHER — POSTGIS

- `ST_ClusterKMeans` — cluster points for heatmaps
- `ST_Hexagon` — hexbin aggregation
- Raster support — elevation, imagery
- `pgbouncer` — connection pooling in production
- Book: *PostGIS In Action* (Manning Press)

GOING FURTHER — MARTIN & MAPLIBRE

- Martin: SQL functions as tile sources; MBTiles for offline; CDN caching
- MapLibre: heatmap layers; 3D terrain; Deck.gl integration; live GeoJSON sources

REFERENCES

PostGIS postgis.net/documentation (<https://postgis.net/documentation/>)

Martin maplibre.org/martin (<https://maplibre.org/martin/>)

MapLibre GL JS maplibre.org/maplibre-gl-js/docs (<https://maplibre.org/maplibre-gl-js/docs/>)

OpenFreeMap openfreemap.org (<https://openfreemap.org/>) — free basemap, no API key

QGIS qgis.org (<https://qgis.org/>) — free desktop GIS

Podman podman.io/docs (https://podman.io/docs)

GitHub: github.com/peaketl/open-source-mapping-presentation
Contact: jason@peaketl.com · (719) 310-5240 · www.peaketl.com